

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

Students will be able to:

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

6. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Output:

Training data shape: (60000,28,28)

Training labels shape:(60000)

Testing data shape: (10000,28,28)

Testing labels shape:(10000)

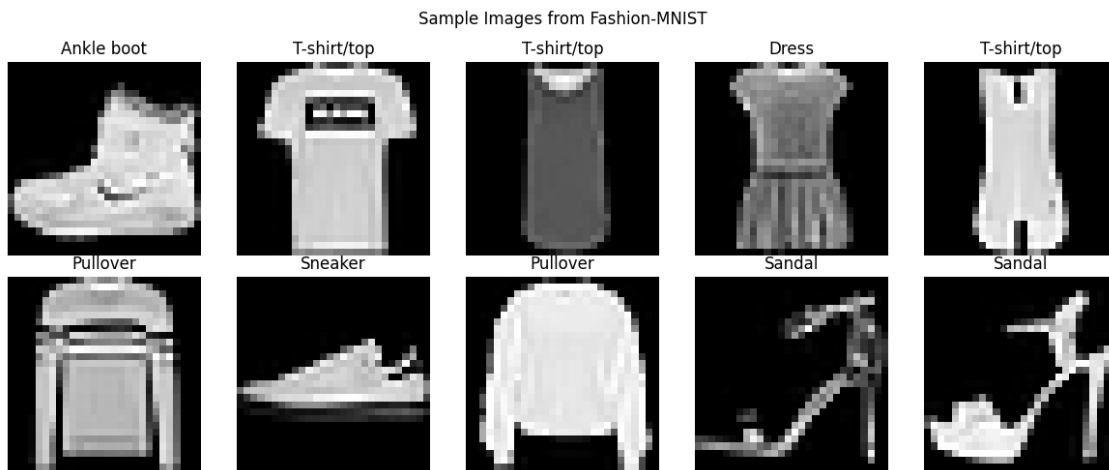


Figure 1: Sample images from the training dataset



Figure 2: Class distribution in the training dataset

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Output:

Shape before preprocessing:

x_train: (60000, 28, 28) x_test: (10000, 28, 28)

Shape after preprocessing:

x_train_flat: (60000, 784) x_test_flat: (10000, 784)

y_train_ohe: (60000, 10) y_test_ohe: (10000, 10)

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Output:

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	100,480
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 10)	650

Total params: 109,386

Trainable params: 109,386

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Output:

Epoch	1/20	accuracy: 0.9381	loss: 0.1618	val_accuracy: 0.8866	val_loss: 0.4048
Epoch	2/20	accuracy: 0.9399	loss: 0.1566	val_accuracy: 0.8847	val_loss: 0.4254
Epoch	3/20	accuracy: 0.9420	loss: 0.1505	val_accuracy: 0.8861	val_loss: 0.4484
Epoch	4/20	accuracy: 0.9444	loss: 0.1464	val_accuracy: 0.8870	val_loss: 0.4549
Epoch	5/20	accuracy: 0.9450	loss: 0.1446	val_accuracy: 0.8802	val_loss: 0.4739
Epoch	6/20	accuracy: 0.9474	loss: 0.1387	val_accuracy: 0.8813	val_loss: 0.4800
Epoch	7/20	accuracy: 0.9474	loss: 0.1364	val_accuracy: 0.8808	val_loss: 0.5172
Epoch	8/20	accuracy: 0.9491	loss: 0.1324	val_accuracy: 0.8800	val_loss: 0.5154
Epoch	9/20	accuracy: 0.9503	loss: 0.1313	val_accuracy: 0.8827	val_loss: 0.4747
Epoch	10/20	accuracy: 0.9501	loss: 0.1271	val_accuracy: 0.8827	val_loss: 0.5169

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

Output:

Accuracy: 0.8744

Precision: 0.8795

Recall: 0.8744

F1-score: 0.8754

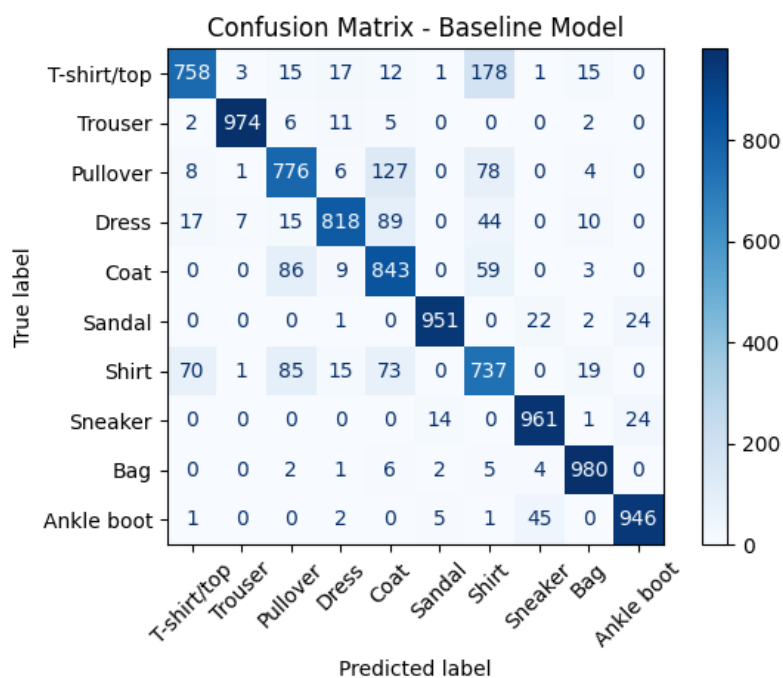


Figure 3: Confusion matrix

Classification Report:

	precision	recall	f1-score	support
T-shirt/top	0.89	0.76	0.82	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.79	0.78	0.78	1000
Dress	0.93	0.82	0.87	1000
Coat	0.73	0.84	0.78	1000
Sandal	0.98	0.95	0.96	1000
Shirt	0.67	0.74	0.70	1000
Sneaker	0.93	0.96	0.95	1000
Bag	0.95	0.98	0.96	1000

Ankle boot	0.95	0.95	0.95	1000
accuracy			0.87	10000
macro avg	0.88	0.87	0.88	10000
weighted avg	0.88	0.87	0.88	10000

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU,Tanh,Sigmoid
Dropout Rate (Optional)	0.0, 0.2

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

Output – Randomized Search:

Fitting 2 folds for each of 5 candidates

Best Score: 0.7775

Best Params: {optimizer: sgd,neurons: 128,learning_rate: 0.1,
hidden_layers: 2,dropout_rate: 0.0,activation:tanh,
epochs: 10,batch_size: 128}

Output- Retraining with Best Hyperparameters:

Epoch	1/10	accuracy: 0.7775	loss: 0.6262	val_accuracy: 0.8267	val_loss: 0.4857
Epoch	2/10	accuracy: 0.8341	loss: 0.4589	val_accuracy: 0.8451	val_loss: 0.4365
Epoch	3/10	accuracy: 0.8486	loss: 0.4186	val_accuracy: 0.8536	val_loss: 0.4135
Epoch	4/10	accuracy: 0.8579	loss: 0.3934	val_accuracy: 0.8572	val_loss: 0.4001
Epoch	5/10	accuracy: 0.8643	loss: 0.3749	val_accuracy: 0.8583	val_loss: 0.3911
Epoch	6/10	accuracy: 0.8693	loss: 0.3602	val_accuracy: 0.8590	val_loss: 0.3845
Epoch	7/10	accuracy: 0.8736	loss: 0.3478	val_accuracy: 0.8605	val_loss: 0.3794
Epoch	8/10	accuracy: 0.8779	loss: 0.3369	val_accuracy: 0.8612	val_loss: 0.3751
Epoch	9/10	accuracy: 0.8816	loss: 0.3272	val_accuracy: 0.8617	val_loss: 0.3713
Epoch	10/10	accuracy: 0.8843	loss: 0.3184	val_accuracy: 0.8631	val_loss: 0.3679

Optimized Accuracy: 0.8538

Optimized Precision: 0.8680

Optimized Recall: 0.8538

Optimized F1-score: 0.8561

Baseline vs Optimized Accuracy: 0.8744 -0.8538

8. Mandatory Plots

Generate the following plots.

- 1. Sample Images
 - 2. Class Distribution
 - 3. Training Accuracy vs Epoch
 - 4. Validation Accuracy vs Epoch
 - 5. Training Loss vs Epoch
 - 6. Validation Loss vs Epoch
 - 7. Confusion Matrix
 - 8. Hyperparameter Search Results
 - 9. Best Model Accuracy Comparison
- Write a 2–3 line inference for each plot.

1. Sample Images

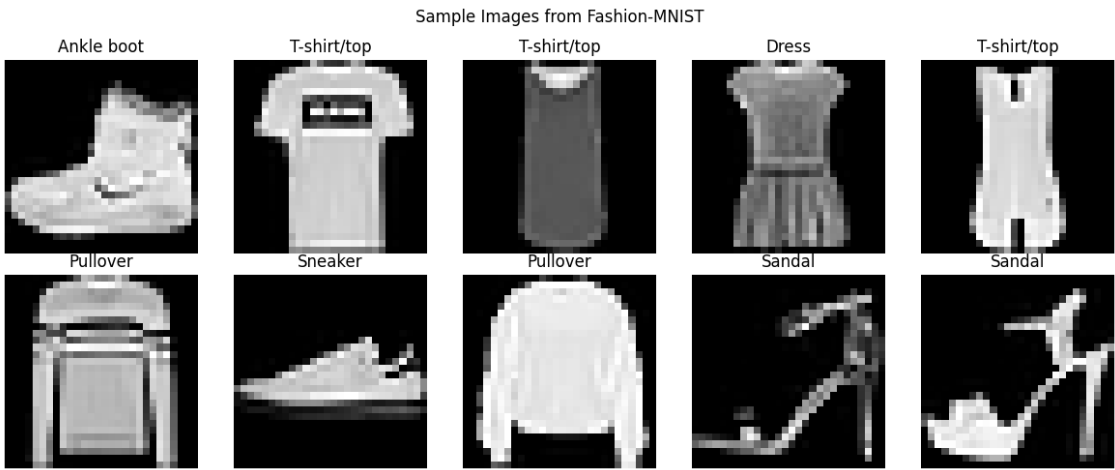


Figure 4: Sample images from Fashion-MNIST

2. Class Distribution



Figure 5: Class distribution across the 10 categories

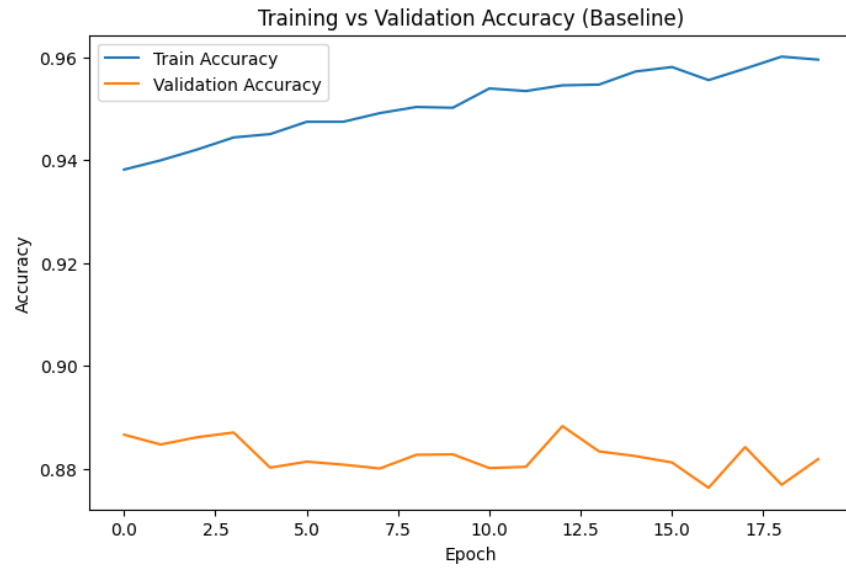


Figure 6: Training vs validation accuracy over epochs (baseline model)

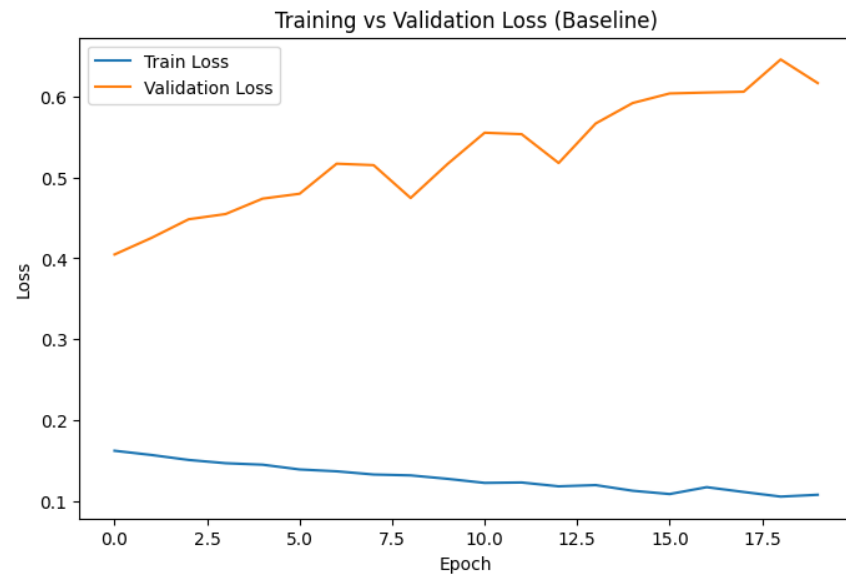


Figure 7: Training vs validation loss over epochs (baseline model)

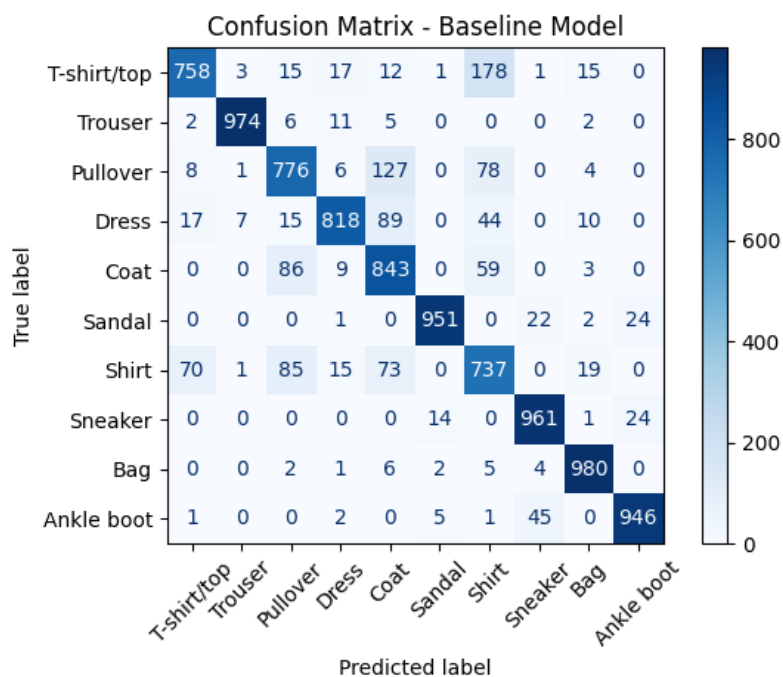


Figure 8: Confusion matrix – baseline model

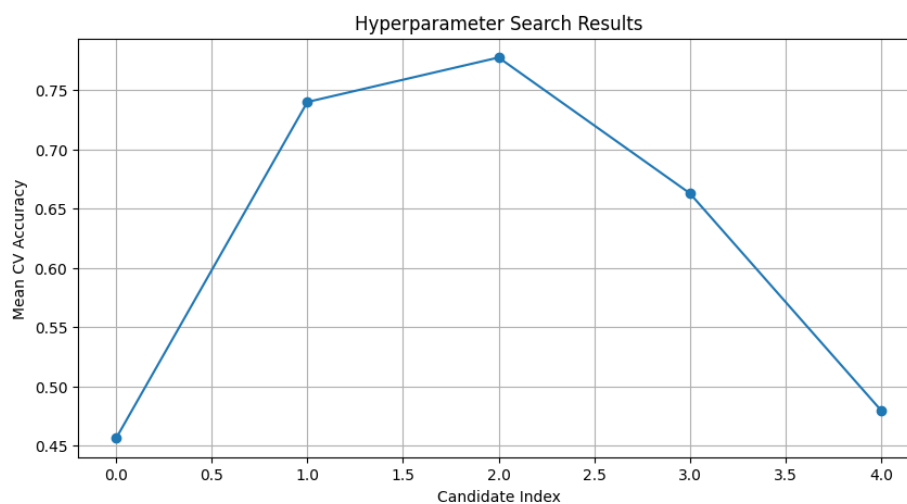


Figure 9: Mean cross-validation accuracy across candidate hyperparameter combinations

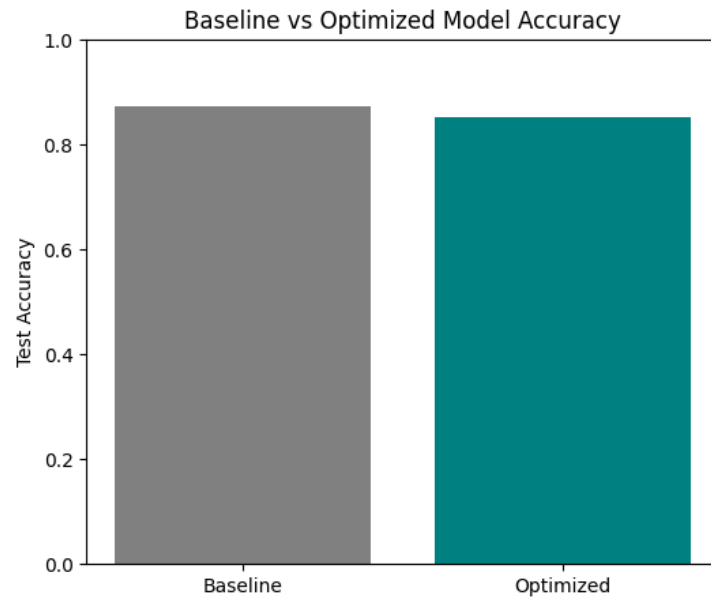


Figure 10: Baseline vs optimized model test accuracy

9. Results

Best Hyperparameters

Hidden Layers	2
Hidden Neurons	128
Learning Rate	0.1
Batch Size	128
Optimizer	SGD
Activation Function	Tanh
Epochs	10
Dropout	0.0
Cross-validation Accuracy	0.7775
Testing Accuracy	0.8538

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8744	0.8538
Precision	0.8795	0.8680
Recall	0.8744	0.8538
F1-score	0.8754	0.8561
Training Time	≈180s (20 epochs)	≈25s (10 epochs)

10. Discussion

Answer the following:

1. Which optimization method was used? Randomized search
2. Which hyperparameters were selected?
3. Did optimization improve performance? No
4. Which hyperparameter had the greatest impact? Adam at default learning rate
5. Compare Grid Search and Randomized Search.
6. Recommend the best MLP configuration.

11. Report Contents

Include Objective, Theory, Dataset, Experimental Procedure, Source Code, Hyperparameter Optimization, Results, Plots with Inference, Discussion, Conclusion and References.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.